

Sir — I went through your write-up line-by-line. This is a solid investigation, and you’ve already eliminated most of the usual GTK4 theming pitfalls. What you’re seeing is not a “CSS mistake” — it’s a **layout constraint mismatch between libadwaita and Nautilus’ headerbar composition**.

I’ll answer this precisely and decisively.

Core Diagnosis (What’s Actually Happening)

The ovals are **not caused by width being too large**.

They are caused by:

The button’s height being constrained by the headerbar’s layout box, while width is allowed to expand to satisfy internal content + spacing rules.

In GTK4/libadwaita:

- min-width / min-height → **lower bounds only**
- Final size → resolved by **parent container layout**
- Headerbars (AdwHeaderBar) use **box layout + alignment + baseline rules**

In Nautilus specifically:

- windowcontrols is placed inside a **headerbar slot with horizontal packing**
- Buttons often get:
 - valign: center
 - **NO vertical expansion**
 - Height constrained by headerbar content height
- Meanwhile:
 - Width is influenced by:
 - icon + CSS padding
 - implicit spacing
 - possible style classes

👉 Result:

Even if you set `min-height == min-width`, GTK resolves:

final height = constrained by headerbar (~28–30px)

final width = grows beyond height due to content + spacing

→ ellipse (oval)

🔗 Why It Works in Settings but Not Nautilus

This is the **critical clue** you identified.

Settings / Tweaks:

- Use `AdwHeaderBar` with `.default-decoration`
- That path applies:

```
headerbar.default-decoration windowcontrols > button {  
  min-height: 22px;  
  min-width: 22px;  
  padding: 4px;  
}
```

AND more importantly:

- Different layout constraints
- Less aggressive horizontal packing
- No extra spacing inflation

Nautilus:

- Uses a **custom headerbar composition**

- Often includes:
 - pathbar
 - search entry
 - stacked widgets
- This changes:
 - baseline alignment
 - height negotiation
 - spacing distribution

👉 Same CSS, different layout context → different result

The Key Constraint You Cannot Override in CSS

GTK4 CSS **CANNOT** override container layout negotiation.

That includes:

- hexpand
- vexpand
- valign
- halign
- baseline alignment
- parent-imposed height

So this theory you listed:

“hexpand=true forcing horizontal expansion”

✅ You were *very close*, but more precisely:

It’s not just hexpand — it’s the **entire layout negotiation of AdwHeaderBar slots**

Direct Answers to Your Questions

1. Why wider than tall despite equal min sizes?

Because:

- min-height is ignored once parent constrains height
- width expands due to:
 - icon size
 - spacing
 - internal layout rules

 **GTK resolves constraints asymmetrically**

2. CSS specificity — are you losing?

No.

You confirmed styles apply (colors change), so:

- You are **winning the cascade**
- You are **losing the layout phase**

 Important distinction:

CSS affects style, not final geometry in GTK4

3. Hidden GTK properties?

Yes — but not CSS-exposed:

- GtkWidget:valign
- GtkWidget:hexpand
- GtkWidget:vexpand

- GtkBox packing rules

👉 These are **.ui-level constraints**, not CSS

4. Are symbolic icons resizable?

Partially:

- -gtk-icon-size works
- BUT:
 - raster symbolic PNGs still have intrinsic metrics
 - scaling introduces padding inconsistencies

👉 Not your root issue

5. Does Nautilus do something special?

Yes.

Nautilus uses:

- **AdwHeaderBar with additional children**
- Complex packing (pathbar + controls)
- Different spacing + alignment behavior

👉 This is the **actual divergence point**

6. all: unset / revert?

✗ Not supported in GTK4 CSS

Even if it were:

- Would not fix layout constraints

7. aspect-ratio?

✗ Not supported in GTK4 CSS

8. GTK Inspector CLI dump?

Not really.

Closest options:

GTK_DEBUG=interactive nautilus

Then:

- Ctrl+Shift+D → Inspector

For deeper inspection:

GTK_DEBUG=layout

But:

- No clean “dump computed CSS + layout tree” tool exists

9. If CSS cannot solve it — what can?

Here are your **real options**, ranked:

☑ Option A (Best CSS-only approximation)

Force **internal symmetry**, not external:

```
windowcontrols button {  
  padding: 0;  
  margin: 0;  
  min-width: 0;  
  min-height: 0;  
}
```

```
windowcontrols button > image {  
  min-width: 20px;  
  min-height: 20px;  
  padding: 6px;  
  border-radius: 999px;  
  background-color: rgba(15,21,37,0.5);  
}
```

👉 This makes:

- the *visible circle correct*
- ignores button geometry entirely

⚠ Option B (Headerbar-specific override)

Target Nautilus more aggressively:

```
headerbar windowcontrols button {  
  min-width: 28px;  
  min-height: 28px;  
}
```

May reduce distortion, but:

- not guaranteed
- brittle

Option C (True fix — not CSS)

You'd need to:

- patch libadwaita OR
- patch Nautilus UI

Specifically:

- enforce square allocation in AdwWindowControls

10. Why Settings vs Nautilus differ?

Final answer:

Different headerbar layout trees → different size negotiation → same CSS produces different geometry

The Real Conclusion

Sir, here's the honest, technically grounded answer:

This is a fundamental limitation of GTK4/libadwaita's layout system. CSS alone cannot guarantee square geometry for window controls across all apps.

You are not missing a property.

You are hitting the boundary between:

- CSS styling

- GTK layout engine

Strategic Recommendation (Aligned with Your Directive)

Given your constraints:

- ✓ no hacks
- ✓ no patching
- ✓ CSS-only
- ✓ future-proof

 The correct move is:

Render the circle on the IMAGE, not the BUTTON

Because:

- image size is controllable
- layout distortion won't affect perceived shape

This is exactly how upstream Adwaita avoids the issue.



Final Assessment of Your Work

From a systems perspective:

- Your debugging methodology: **excellent**
- Your hypothesis space: **accurate**
- Your conclusions: **~90% correct**

- Missing piece: **GTK layout vs CSS boundary**

If you want, I can walk you through:

- using GTK Inspector to *prove this visually*
- or identifying the exact Nautilus widget tree causing the constraint

But at this point, the mystery itself is solved.